TELASIST × INVEX · ALINEACIÓN TÉCNICA SKALA

Matriz de preguntas para Jonatas

Preguntas con contexto detallado sobre lo encontrado en el sandbox del 12-Oct. 9 ítems ordenados por tipo: 2 convenciones a decidir, 5 documentación/clarificación, 2 cierre operativo. No bloquean el avance, pero destraban el code review conjunto.



PARA

Jonatas Amorim · Skala

BASE

Audit sandbox 12-Oct · AF_Voice_Invex_Retenciones_TDC v5

FORMATO

1:1 técnica o async en comments

01 · CONVENCIONES A DECIDIR

2 preguntas que afectan el código

Diseño que necesitamos cristalizar antes del code review conjunto para que las dos cabezas (Salesforce + Skala) sigan el mismo criterio.

Q1 · Duplicación de nombres de acciones en el DSL

CONVENCIÓN

¿Las dos variantes de nombre por acción (`Validar_tarjetahabiente` vs `validar_tarjetahabiente`) son intencionales?

CONTEXTO OBSERVADO

Al parsear el DSL del Retenciones aparecen 7 pares de referencias con la misma raíz pero casing distinto:

```
@actions.Validar_tarjetahabiente   ·  @actions.validar_tarjetahabiente
@actions.Obtener_oferta            ·  @actions.obtener_oferta
@actions.Siguiente_oferta          ·  @actions.siguiente_oferta
@actions.Registrar_resultado       ·  @actions.registrar_cancelacion / registrar_retencion /
registrar_datos
@actions.Registrar_escalamiento    ·  @actions.marcar_escalamiento
@actions.Transferir_a_humano       ·  @actions.transferir
@actions.Finalizar_llamada         ·  @actions.finalizar_llamada
@actions.confirmar_cancelacion     ·  (solo lowercase)
@actions.confirmar_rfc             ·  (solo lowercase)
```

HIPÓTESIS

Dos opciones plausibles: (a) las variantes **Capital_Case** son las acciones públicas (invocable Apex o Flow), las **lowercase** son triggers internos que el DSL usa como alias/estado. (b) Es duplicación por iteración rápida sin cleanup. Si es (a), vale la pena documentar la convención; si es (b), consolidar antes del merge.

Impacto si no se decide: code review más lento · deuda que crece con cada iteración.

Esfuerzo: 10 min decisión + 1-2 horas limpieza o docs.

Q2 · Separación de "siguiente oferta" como acción propia

CONVENCIÓN

¿`InvexSiguienteOferta` es una acción pública real o es un wrapper muy delgado?

CONTEXTO OBSERVADO

La clase `InvexSiguienteOferta` pesa solo **393 caracteres** (comparado con `InvexObtenerOferta` de 5,108). Al tenerla como acción propia expuesta al agente permite rotación de ofertas en el manejo de objeciones – bien pensado. Pero si solo delega al Engine, convendría documentar que es un "alias semántico" para el prompt y no una ruta de código distinta.

HIPÓTESIS

Es un wrapper deliberado para que el DSL tenga dos verbos distintos (`Obtener_oferta` inicial · `Siguiente_oferta` en rotación) sin que el agente tenga que llevar estado de "cuál número de oferta vamos". El estado vive en el motor. Confirmemos y lo documentamos como patrón.

Impacto: claridad del contrato que ve el agente · facilita futuro mantenimiento. **Esfuerzo:** 5 min + 1 línea de comentario en Apex.

02 · DOCUMENTACIÓN · CLARIFICACIÓN

5 preguntas que destraban el runbook y el code review

Q3 · Dónde vive la lógica de folio RB/CC/CB + DDMMAA + últ.4

DOCUMENTACIÓN

¿La construcción del folio está dentro de `InvexCaseWriter` o en `InvexRegistrarResultado`?

CONTEXTO

En el plan original estaba `Build_Folio` como acción invocable separada. En la implementación no aparece una clase con ese nombre. El folio tiene formato **prefijo (RB/CC/CB) + fecha DDMMAA + últimos 4 dígitos de la tarjeta**. Si vive dentro de `InvexCaseWriter.buildFolio()` está bien – necesitamos documentarlo para el runbook de F4 y para la QA del piloto que necesita parsear folios para medir mix de outcomes.

Impacto: dashboard A/B necesita saber dónde parsear. **Esfuerzo:** confirmar en 2 min · 1 párrafo en el runbook.

Q4 · Comentario a planchar · TYPE|CAT1|CAT2|CAT3|CAT4|COMMENT

DOCUMENTACIÓN

¿Dónde arma el pipe-concat del `TIPIFICADOR_COMENTARIO_A_PLANCHAR__c`?

CONTEXTO

El Case tiene un campo textarea donde se persiste el "resumen a planchar" con el formato `TYPE|CAT1|CAT2|CAT3|CAT4|COMENTARIO ≤150 chars`. En el audit vimos un ejemplo real: `TEAT||||TH SEGUIMIENTO DE ANUALIDAD NO APLICADA RB290526`. En el plan era `Build_Analyst_Comment`. Asumimos que está en `InvexCaseWriter` o en `InvexRegistrarResultado` · misma pregunta de documentación que Q3.

Impacto: QA del piloto necesita validar que el formato se respeta. **Esfuerzo:** confirmar en 2 min.

Q5 · Baseline Águila · propósito actual y futuro

DOCUMENTACIÓN

¿Para qué se mantiene `AF_Voice_Invex_Baseline_Aguila_1`?

CONTEXTO OBSERVADO

Dos agentes en el sandbox: `AF_Voice_Invex_Retenciones_TDC_5` (54 KB · 6 subagents · Claude Opus 4.7 · 42 variables) y `AF_Voice_Invex_Baseline_Aguila_1` (22 KB · 1 subagent Retencion_TDC + 4 estándar · **DefaultEinsteinHyperClassifier** · 0 variables).

HIPÓTESIS

El Baseline parece el scaffold inicial clonado de El Águila antes de inyectar lógica. El modelo clasificador no está diseñado para generación – útil solo para pruebas de routing de topics. Posibilidades: (a) línea base para A/B comparación de performance; (b) sandbox para experimentar nuevos topics sin romper el productivo; (c) archivarlo porque ya cumplió su rol de scaffold.

Impacto: si se mantiene, documentar propósito; si no, archivar para no confundir al equipo. **Esfuerzo:** 10 min decisión + acción.

Q6 · Modelo elegido · Claude Opus 4.7 vs otros

DOCUMENTACIÓN

¿Se consideraron GPT-5 o modelos más chicos para el Retenciones? ¿Por qué Opus?

CONTEXTO

El Retenciones usa `sfdc_ai__DefaultBedrockAnthropicClaude47Opus`. Opus es el modelo más capaz pero también el más costoso por token. El flujo tiene instrucciones muy explícitas y lógica determinista – en teoría un modelo más chico podría funcionar. Útil documentar la decisión para la conversación de costos del rollout 100 % (~340k conversaciones/año).

Impacto: presupuesto en GA · pre-aprobación. **Esfuerzo:** 15 min para capturar racional y comparativa.

Q7 · Cobertura del Repository (80 %) y Service (91 %)

DOCUMENTACIÓN

¿Las líneas no cubiertas son casos de error legítimos o quedó por iterar?

CONTEXTO

Las 5 acciones invocables están al 100 %, pero `InvexRetencionRepository` queda en 80 % (líneas 52, 57, 75, 127, 128…) y `InvexRetencionService` en 91 % (153, 191, 257–259…). Para producción típicamente queremos ≥ 85 % y entender qué queda sin cubrir. Si son branches de catch/degradation, podría estar bien; si es lógica real sin tests, agregamos antes del deploy.

Impacto: deploy a prod requiere ≥ 75 % orgwide (hoy 17 %, el proyecto solo mueve la aguja parcialmente). **Esfuerzo:** ~2 horas para revisión y top-up si aplica.

03 · CIERRE OPERATIVO

2 preguntas para cerrar F1 formalmente

Q8 · Suite de pruebas Agent Script (`sf agent test`)

OPERATIVO

¿Hay un plan para generar el test spec del agente (`AiEvaluationDefinition`)?

CONTEXTO

Hoy hay 61 pruebas Apex excelentes (100 % pass), pero no existe test spec que corra sobre el DSL end-to-end. El plan F1 incluye 12 escenarios (actividad 1.15): retención exitosa por cada motivo × 5, cancelación sin saldo, pre-cancelación, fallo de identidad, cliente no encontrado, muerte, EXPI, cliente agresivo, beneficio no aplicado, interrupción. Se generan con `sf agent generate test-spec` y se corren con `sf agent test run`.

PROPUESTA

FDE arma el test spec con los 12 escenarios base; Skala extiende con los casos que surgieron durante la implementación. Lo corremos antes del code review conjunto. Es la actividad 1.15 del plan (hoy en 50 %).

Impacto: gate del F1 cerrado · gate del F1.5 abierto. **Esfuerzo:** 2 días conjunto.

Q9 · Preview en vivo con llamada real · validación end-to-end

OPERATIVO

¿El agente ya atendió una llamada real desde el número Partner Voice Forwarding?

CONTEXTO

El bundle v5 está desplegado y el `sf agent validate` está en verde (asumimos). La actividad 1.17 del plan requiere **1 llamada real** (no solo preview text) desde el número forward que Telasist tiene asignado. Esto cierra el loop end-to-end: Amazon Connect → HVCC → Agentforce Voice → Apex actions → Case escrito → folio dictado → VoiceCall grabado.

PROPUESTA

Coordinar con Telasist IT una ventana de 1 hora para hacer 3 llamadas reales de prueba: una retención exitosa, una cancelación sin saldo, una escalación forzada. Grabamos todo (sandbox) y hacemos post-mortem conjunto para enlistar hallazgos finales antes del code review.

Impacto: cierre formal de F1 · demo cerrado para la próxima reunión. **Esfuerzo:** 1 hora + 2 horas post-mortem.

Cómo responder. Lo más rápido es resolverlas en la 1:1 técnica (30 min bastan para las 9). Si preferís async, por favor dejá nota en el Case del portal o en el canal Slack del proyecto. Las 2 urgentes (Q8, Q9) necesitamos cerrarlas antes del 24-oct para no mover la fecha de F1.5.